

CodeLeWM: A Reproducible, Claim-Gated Code World-Model Study with Negative Action-Use Results and a Narrow Downstream Reranking Slice

Abdelhamid Bakhta
StarkWare

<https://github.com/AbdelStark/CodeLeWM>

Draft of June 9, 2026

Abstract

We study whether a JEPA-style latent code world-model can rerank candidate program edits without executing them, scoring candidates by predicted latent transitions rather than by generating patches. We release CodeLeWM, a manifest-backed research artifact—a ~ 45 M-parameter transition model with an EMA target encoder and LeJEPA-family SIGReg anti-collapse—and evaluate it under a *pre-registered*, control-first claim gate: the acceptance thresholds and the no-action, lexical, and shuffled-action controls are fixed in the training configuration before any run, and a machine-readable gate decides which claims may open. The outcome is one falsified hypothesis, healthy diagnostics, and a single narrow downstream win that does not open an aggregate claim. The original commit-edit action-use hypothesis is negative: on 1,000 held-out retrieval examples, commit-edit text-action ranking (Recall@1 0.263, MRR 0.370) loses to a no-action baseline (0.441, 0.533). A separate execution-trace substrate is healthier in diagnosis—execution retrieval beats every control (Recall@1 0.254/0.273 vs. all controls below 0.024) and single-point mutation surprise reaches AUC 1.0 over 260 scored pairs—but these are training-health signals, not downstream utility. The only open downstream slice is HumanEval WS-D, a deterministic mutation-distractor reranking pack of $n = 47$ problems: CodeLeWM reaches 46/47 (0.979) on both seeds, beating the strongest non-trivial control, shuffled-action (0.894/0.915), by +8.5 and +6.4 points (4 and 3 problems; 95% bootstrap CIs [0.0, 19.1] and [0.0, 14.9] that *include* zero), and the no-action control by +10.6 and +8.5 points. The two seeds are a training-stability check on the same 47 problems, not independent replications. The aggregate model-quality claim stays closed: MBPP-Plus WS-D is saturated on its 17 problems—CodeLeWM, no-action, and lexical all reach 17/17, so the slice has zero headroom and is non-informative for any lift—representation probes do not beat controls, broad semantic surprise has no scorable pairs, and the downstream rows expose no standalone `p_pass` score. We therefore present CodeLeWM as a reproducible negative-and-diagnostic study with one narrow positive slice, not as evidence that the model generally improves coding, together with a released anti-saturation falsification harness whose gate is deliberately still closed. The contribution is the artifact and the discipline: code-world-model reranking claims need no-action, lexical, and shuffled-action controls and pre-registered gates because many apparent wins vanish under those controls.

1 Introduction

World models are appealing for code because code edits have a natural transition structure: a before-state, an action or instruction, and an after-state. If a model can learn that transition, it may be useful even when it is not a code generator. A latent scorer could rank candidate patches

produced by humans, search, or large language models, while keeping candidate code untrusted and avoiding execution in the scoring path.

Joint-embedding predictive architectures (JEPAs) learn by predicting future latents rather than reconstructing observations [2, 6]. LeWorldModel studies stable end-to-end JEPA-style world modeling from pixels [17]. CodeLeWM asks whether the same idea can be made credible for code edits. The transition contract is:

$$\hat{z}_{\text{after}} = P(E(s_{\text{before}}), A(a)). \quad (1)$$

The intended downstream use is ranking, not generation: given a before-state, an edit or execution action, and candidate after-states, score which candidate best matches the learned transition.

This paper reports the full evidence arc instead of selecting only the strongest slice. The early commit-message-conditioned substrate fails the action-use gate. A later execution-trace substrate repairs collapse and produces useful diagnostics. The final v0.9/v1.0 downstream replay shows a narrow HumanEval WS-D win but closes the aggregate claim because MBPP-Plus is solved by controls. We frame the result as an artifact-backed negative study: the system makes the tested hypothesis falsifiable, identifies where the model fails, isolates a narrow positive slice, and turns the failure modes into concrete next research requirements.

Research questions.

1. **RQ1: Does action conditioning improve code-transition retrieval on commit-edit data?** No. The v0.2 action-swap intervention loses to no-action on the headline retrieval gate.
2. **RQ2: Can an execution-trace substrate support non-collapsed learned diagnostics?** Yes, in the narrow diagnostic sense. v0.9 retrieval beats retrieval controls, and mutation surprise AUC is 1.0 on both seeds.
3. **RQ3: Does the learned world model improve downstream candidate reranking?** Partially. HumanEval WS-D improves over no-action on two seeds, but MBPP-Plus is saturated by no-action and lexical controls, so the aggregate downstream claim is closed.

Contributions.

1. A *pre-registered, control-first claim-gate methodology* for code-world-model reranking: falsification gates (no-action, lexical, shuffled-action, and LLM-order controls; collapse, retrieval, surprise, and downstream-lift thresholds with two required seeds) are fixed in the training configuration before any run and enforced as a machine-readable artifact contract, so that apparent wins that vanish under controls are caught automatically rather than reported.
2. A reproducible code world-model artifact instantiating this methodology, with dataset construction, training, retrieval, surprise, latent-probe, downstream reranking, paper-demo replay, manifests, checksums, and secret scans.
3. A negative action-use result showing that commit-edit action text does not beat a no-action baseline under the tested intervention, and a diagnostic execution-substrate study showing non-collapse, retrieval lift over controls, and mutation-surprise separation without overstating semantic representation claims.
4. A final downstream replay showing a single non-aggregated HumanEval WS-D slice ($n = 47$) that beats no-action and the stronger shuffled-action control by a few problems, while the pre-registered gate closes the aggregate claim on MBPP-Plus saturation, missing standalone `p_pass` scores, and broader representation and semantic gates.

5. A *released* anti-saturation falsification harness that operationalizes the outlook—a hard-negative pool generator and a three-baseline gate over a locked ≥ 100 -problem slice—whose claim gate remains deliberately closed because no eligible slice has yet been scored.

2 Artifact and Claim Discipline

CodeLeWM is intentionally packaged as a research artifact. Each publishable surface writes a schema-versioned report, manifest, checksum evidence, and secret-scan result. Candidate code, generated patches, configs, checkpoints, and provider outputs are treated as untrusted inputs. The model-scoring path does not import, execute, or test-run candidate patches. Execution labels used for training or benchmark construction are produced in bounded sandboxed data build steps and then consumed as recorded artifacts.

The final paper-demo contract is deterministic. WS-D denotes the mutation-distractor reranking benchmark: for each problem, a known-correct reference solution is paired with lexically similar single-point AST mutants, then labeled by the existing sandbox pipeline and shuffled so reranking has pass/fail headroom. The v1.0 replay consumes checked-in v0.9 WS-D completion labels and score rows for four slices: seed 42 and seed 1729 crossed with HumanEval WS-D and MBPP-Plus WS-D. It covers 128 problem-slice rows and 768 candidate completions. These are 64 unique problems (47 HumanEval and 17 MBPP-Plus) scored under two seeds, at six candidates per problem; the count is replay coverage, not independent problems. Replay mode records `score_source=replay_existing_scores`; it is not a fresh checkpoint scoring run and not a live OpenRouter demo. The committed artifact id is `demo_report-e6fc06c328eed245`.

The release includes *pre-registered*, machine-readable claim gates. The numeric thresholds—collapse effective-rank ratio ≥ 0.20 (achieved 0.388/0.406); mutation-surprise AUC ≥ 0.65 (achieved 1.0/1.0); retrieval Recall@1 lift over no-action ≥ 0.05 (achieved); downstream rerank pass@1 lift ≥ 3.0 points over no-action (HumanEval WS-D passes, MBPP-Plus WS-D +0.00 fails); two required seeds—were committed in the training configuration before the runs (Section 3), so a passing result cannot be retrofitted to the data. The gate is not a decorative checklist; it is part of the artifact contract. It allows narrow HumanEval WS-D wording and blocks aggregate coding-improvement wording because the cross-benchmark gate does not pass.

3 Model and Training

CodeLeWM instantiates the transition contract of Equation (1) with a single module family of approximately 45M parameters (`TorchCodeTransitionModel`). A state encoder E maps a bounded token capsule (state length 1024) to a latent $z \in \mathbb{R}^{256}$ using a four-layer, eight-head Transformer with AdaLN-zero conditioning. An action encoder A maps the textual action (length 256) to an action embedding, and a predictor P maps $(E(s_{\text{before}}), A(a))$ to the predicted after-latent $\hat{z}_{\text{after}}$. Prediction targets are produced by an exponential-moving-average copy $\bar{E}$ of the encoder (decay 0.99), so the objective predicts in latent space and never reconstructs target tokens, in the style of feature-prediction JEPAs [2, 6].

The execution-substrate objective is a weighted sum of a latent prediction term, an anti-collapse term, two action-use terms, an in-batch retrieval term, and two supervised heads:

$$\mathcal{L} = \|\hat{z}_{\text{after}} - \bar{E}(s_{\text{after}})\|_2^2 + \lambda_{\text{sig}}\mathcal{L}_{\text{SIGReg}} + \lambda_{\text{swap}}\mathcal{L}_{\text{swap}} + \lambda_{\text{inv}}\mathcal{L}_{\text{inv}} + \lambda_{\text{ret}}\mathcal{L}_{\text{ret}} + \lambda_{\text{pass}}\mathcal{L}_{\text{pass}} + \lambda_{\text{val}}\mathcal{L}_{\text{val}}, \quad (2)$$

with weights $\lambda_{\text{sig}}=0.09$, $\lambda_{\text{swap}}=0.1$, $\lambda_{\text{inv}}=0.05$, $\lambda_{\text{ret}}=0.05$, $\lambda_{\text{pass}}=0.5$, and $\lambda_{\text{val}}=0.2$. SIGReg is the sketched isotropic-Gaussian (Epps–Pulley) regularizer of the LeJEPa family [4, 17]; it is the anti-

collapse term. $\mathcal{L}_{\text{swap}}$ penalizes predictions under a mismatched action, $\mathcal{L}_{\text{inv}}$ asks the latent transition to retain action information, and $\mathcal{L}_{\text{ret}}$ is an in-batch InfoNCE-style retrieval term. The two auxiliary heads are a correctness head trained with a class-balanced **p_pass** BCE loss (positive weight 1.024) and an output-value cross-entropy head; we treat both as training-time auxiliary signals, not as standalone downstream scores.

We optimize with AdamW (learning rate 3×10^{-4} , $\beta = (0.9, 0.95)$, weight decay 0.1) at an effective batch size of 256 (micro-batch 64, gradient accumulation 4) for 12,000 steps with 500 warmup steps, cosine decay to zero, and gradient clipping at 1.0, in **bf16** mixed precision on a single A10G GPU. Every configuration is trained for two seeds, 42 and 1729. At rerank time the model is used purely as a scorer: a candidate after-state s_{cand} is embedded and ranked by its negated transition energy $-\|\hat{z}_{\text{after}} - E(s_{\text{cand}})\|_2^2$ (lower latent distance ranks higher). Candidate code is never imported, executed, or test-run on the scoring path.

Pre-registered acceptance gates. The training configuration fixes the acceptance thresholds *before* the runs, not after inspecting them. Non-collapse requires a predicted-latent effective-rank ratio above 0.20; mutation-surprise AUC must exceed 0.65; retrieval Recall@1 must exceed no-action by at least 0.05; a downstream slice may open only with a pass@1 lift of at least 3.0 points over no-action; and every gate requires both seeds as a training-stability check. These thresholds, rather than a post-hoc reading of the metrics, decide which claims open.

4 Data

The execution pack (`codelewm-execution-pack@v0.9.0-rc1`) contains 2,188 sandbox-labeled execution transitions built from HumanEval (1,882 records) and MBPP-Plus (306 records) completions, split 1,928/57/203 into train/validation/test. Pass/fail labels are near-balanced (1,081 passing, 1,107 failing; positive weight 1.024); 26 records were dropped for sandbox timeouts. Labels are produced by a bounded, allowlisted sandbox-execution data-build step and then consumed as recorded artifacts; sandbox execution is never part of training or scoring. The HumanEval and MBPP-Plus WS-D reranking slices used in Section 5.3 are held out from training. Each WS-D problem supplies six candidates (1 reference plus AST mutants), giving 47 HumanEval and 17 MBPP-Plus test problems per seed.

The two seeds vary only model initialization and training order; both are evaluated on the *identical* 47-problem (HumanEval) and 17-problem (MBPP-Plus) WS-D sets with identical candidate pools. Cross-seed agreement is therefore evidence of training stability, not of independent statistical replication, and the per-seed confidence intervals are correlated because they resample the same problems. We treat the pre-registered two-seed requirement as a stability check accordingly.

5 Experimental Evidence

The final claim audit is `docs/benchmark/V1_0_FINAL_CLAIM_AUDIT_2026-06-08.md`. It consolidates v0.2, v0.6, v0.8, v0.9, and v1.0 paper-demo evidence. Table 1 summarizes the arc.

5.1 RQ1: negative action-use evidence

The v0.2 action-swap/inverse-action run directly tests whether action conditioning beats no-action on held-out retrieval. It does not. Text-action Recall@1 was 0.263 and MRR 0.370 on 1,000 held-out retrieval examples, while no-action reached Recall@1 0.441 and MRR 0.533, a gap far outside

Table 1: Evidence arc and public claim status. Each milestone is useful, but only the listed claims are open.

Milestone	Main evidence	Open claim	Closed or blocked claim
v0.2	action-swap A10G run	systems pipeline	action-use, latent axes, downstream utility
v0.6	execution substrate	diagnostics	stable downstream HumanEval/MBPP utility
v0.8	execution repair	HumanEval WS-D slice	MBPP, magnitude probe, broad semantic gate
v0.9	two-seed final gate suite	HumanEval WS-D slice	MBPP, representation, broad semantic gate
v1.0	paper-demo replay	reproducible demo package	aggregate downstream usefulness

sampling noise at this n . Exact-same-before, near-before, same-file, action-cluster, edit-shape, and random action-contrast pools also failed to show the required margin over no-action. Latent probes were unsupported for semantic structure: lexical or metadata controls beat the latent views on the available targets.

This is the central negative result. It means that the early edit-action signal was not sufficiently action-discriminative. A commit message can describe a change, but it often does not force the model to use the action rather than the before-state. The no-action baseline is therefore not a nuisance baseline; it is a first-class scientific control.

5.2 RQ2: execution-substrate diagnostics

The execution substrate changes the state/action/next-state signal from noisy commit messages to deterministic execution triples. This improves training health and diagnostic separability. By v0.9, retrieval lift over controls passes on both seeds: Recall@1 is 0.2538 and 0.2731, while all listed retrieval controls are below 0.024 Recall@1. Mutation surprise AUC is 1.0 on both seeds, over 260 scored single-point mutation pairs; the broader semantic-decoy categories have zero scorable aligned pairs, so this is narrow mutation surprise, not broad semantic surprise. These are credible training-health results. They do not, by themselves, prove downstream coding usefulness.

Representation claims remain closed. The v0.9 **passed** probe reaches 0.6552 and 0.6158 for $\hat{z}_{\text{after}}$, but no-action or before-state controls are stronger. The **output_magnitude_bucket** target is evaluable in v0.9 but tied with a direct **z_after** control. Broad semantic surprise also remains closed because the semantic-decoy categories have zero scorable aligned pairs in the final pack.

5.3 RQ3: final downstream replay

The final paper-demo replay is the central downstream result. It is a clean-checkout replay over persisted v0.9 score rows, not a new scoring run, and is derived from `docs/benchmark/v1_0/paper_demo/reports/paper_demo_report.json` and summarized in Figure 1 and Table 2.

HumanEval WS-D is the narrow positive slice. CodeLeWM ranks a passing candidate first on 46/47 problems (pass@1 0.9787) on both seeds, beating no-action by +10.64 and +8.51 points—that is, 5 and 4 of 47 problems—with 95% paired-bootstrap CIs of [2.13, 21.28] and [2.13, 17.02] (Table 3). It also beats the stronger shuffled-action control, which supplies a real but mismatched action (pass@1 0.8936 and 0.9149), by +8.51 and +6.38 points (4 and 3 problems). Because shuffled-action dominates no-action on exactly these slices, the win over shuffled-action—not over

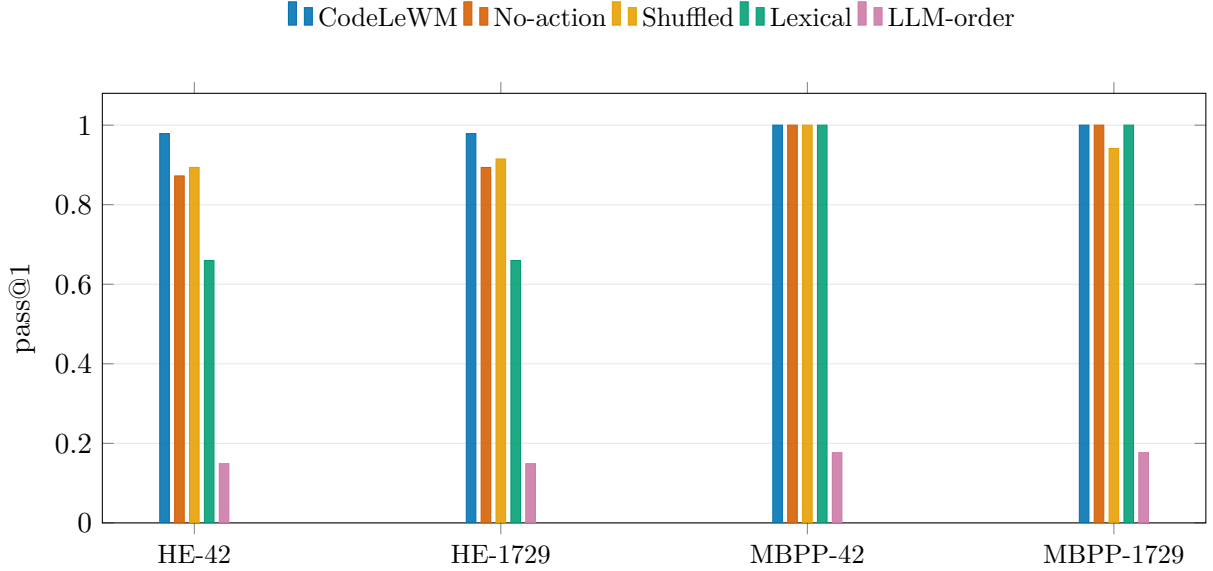


Figure 1: Final v1.0 paper-demo replay ($n = 47$ HumanEval, $n = 17$ MBPP-Plus problems per seed). HumanEval WS-D shows a narrow CodeLeWM lift over both the no-action and the stronger shuffled-action control (their bars nearly coincide, just below CodeLeWM; the lift over shuffled-action has a 95% CI that includes zero, Table 3), while MBPP-Plus is saturated by CodeLeWM, no-action, and lexical controls. This is why the aggregate claim gate is closed.

Table 2: Final v1.0 paper-demo replay (HumanEval WS-D $n = 47$, MBPP-Plus WS-D $n = 17$ problems per seed). The aggregate claim gate is closed even though the HumanEval WS-D slices are open. *Lift* is pass@1 points over no-action; Table 3 adds the stronger shuffled-action control and the problem-count breakdown.

Seed	Benchmark	CodeLeWM	No-action	LLM-order	Lexical	Lift	Gate
42	HumanEval WS-D	0.9787	0.8723	0.1489	0.6596	+10.64	open
42	MBPP-Plus WS-D	1.0000	1.0000	0.1765	1.0000	+0.00	closed
1729	HumanEval WS-D	0.9787	0.8936	0.1489	0.6596	+8.51	open
1729	MBPP-Plus WS-D	1.0000	1.0000	0.1765	1.0000	+0.00	closed

no-action—is the binding result: the lift is not explained by the mere presence of an action token. But against this strongest control the 95% CIs are $[0.00, 19.15]$ and $[0.00, 14.89]$, which *include* zero, so even the open slice is suggestive, not robust. The intervals are coarse: on $n = 47$ the smallest resolvable lift is one problem (2.13 points), the CIs are percentile bootstraps without bias correction, and the two seeds reuse the same 47 problems. LLM-order is far weaker at pass@1 0.1489.

MBPP-Plus is the counterweight. On its 17 problems, CodeLeWM, no-action, and lexical all rank a passing candidate first (pass@1 1.0000, i.e. 17/17), so the slice has zero headroom and is non-informative for any lift: the $[0.00, 0.00]$ bootstrap interval reflects the absence of variation in a saturated 17-problem vector, not an equivalence result. This is exactly the configuration the pre-registered gate refuses to aggregate, so it closes the aggregate downstream claim. Across the full evaluation—two benchmarks $\times$ two seeds $\times$ four controls—we apply no multiple-comparison correction; the single surviving open slice should be read as a hypothesis-generating result that cleared a pre-registered gate, not as a multiplicity-controlled confirmation. The evidence supports

Table 3: HumanEval WS-D control breakdown ($n = 47$ problems, six candidates each; seeds 42/1729). CodeLeWM ranks a passing candidate first on 46/47 problems on both seeds. *Lift* is in pass@1 points (problem count in parentheses); 95% CIs are paired problem-level percentile bootstraps (Section 5.3); the no-action intervals exactly reproduce the released values, and the shuffled-action intervals are recomputed here with the same procedure (seed 17, 2000 resamples). Random is the trivial floor. Controls are ordered weakest to strongest: shuffled-action is the strongest non-trivial control, and the CodeLeWM lift over it includes zero.

Control	pass@1 (count/47)		lift (problems)		95% bootstrap CI	
	s.42	s.1729	s.42	s.1729	s.42	s.1729
LLM-order	7/47	7/47	+83.0 (39)	+83.0 (39)	—	—
Random	10/47	10/47	+76.6 (36)	+76.6 (36)	—	—
Lexical	31/47	31/47	+31.9 (15)	+31.9 (15)	—	—
No-action	41/47	42/47	+10.6 (5)	+8.5 (4)	[2.13, 21.28]	[2.13, 17.02]
Shuffled-action	42/47	43/47	+8.5 (4)	+6.4 (3)	[0.00, 19.15]	[0.00, 14.89]

a narrow HumanEval WS-D result, not a claim that CodeLeWM generally improves patch ranking.

6 Lessons

No-action is a scientific baseline, not a convenience baseline. Many code transitions are sparse. Large portions of the after-state are identical to the before-state, and many candidate completions can be ranked using surface similarity. A world model must learn when the action changes behavior, not merely when text resembles the prompt. The v0.2 failure shows that weak action text can leave no-action stronger than the learned transition.

Substrate health is not downstream utility. The execution substrate yields healthier retrieval and mutation diagnostics, but those diagnostics do not automatically imply useful patch ranking. This distinction matters. Non-collapse, retrieval lift over controls, and surprise separation are necessary health signals, not sufficient product claims.

Benchmark saturation can close an otherwise promising result. The final WS-D MBPP-Plus pack is saturated on the available 17 problems: CodeLeWM, no-action, and lexical all choose passing candidates at rank 1 on both seeds, so the slice has no headroom and cannot register a lift. This is not a failure of the replay command, and it is not evidence of equivalence on a larger benchmark; it is precisely the zero-headroom case the claim gate is designed to refuse.

Representation claims require their own evidence. Healthy training diagnostics do not justify naming semantic latent axes. Frozen probes must beat controls across targets and seeds, and semantic-decoy surprise must have enough scorable aligned pairs. In the final evidence, those conditions do not hold.

7 Related Work

Joint-embedding prediction. I-JEPA and V-JEPA learn feature predictions without pixel reconstruction [2, 6]. LeWorldModel studies stable end-to-end JEPA-style world modeling from pixels

[17]. Anti-collapse regularization is central to this family. CodeLeWM adopts the LeJEPA-family Sketch Isotropic Gaussian Regularizer (SIGReg), an Epps–Pulley-statistic penalty that drives embeddings toward an isotropic Gaussian, rather than the variance–covariance penalty of VICReg or the cross-correlation penalty of Barlow Twins [4, 5, 23]; SIGReg alone keeps the predicted-latent effective-rank ratio above the 0.20 gate (0.388/0.406 across seeds).

Code modeling and execution signals. Pretrained code models such as CodeBERT, GraphCodeBERT, UniXcoder, and CodeT5 learn program representations or generation models from code and natural language [10–12, 21]. HumanEval, MBPP, APPS, CodeNet, and EvalPlus-style benchmark extensions provide candidate tasks and labels for coding evaluation [3, 8, 13, 16, 18]. Generated tests have also been used for reranking code generations [7]. Execution-aware pretraining and execution-trace work provide complementary signals for program behavior [1, 9, 22]. Architecturally, CodeLeWM is an incremental port of LeJEPA-family anti-collapse and EMA-target JEPA prediction to code-execution transitions. Its methodological contribution is distinct: pre-registered, machine-readable claim gates with first-class no-action, lexical, and shuffled-action controls, so that reranking claims are falsifiable before they are reported.

Reranking and scoring candidate code. A second line, directly relevant to how CodeLeWM is used, ranks or filters sampled candidate programs rather than generating them. Self-consistency and sample clustering select among many decoded samples by agreement [15, 20]; test-based reranking such as CodeT uses generated tests [7]; learned critics and likelihood-based reviewers score completions for functional correctness [14, 24]; and execution-based minimum-Bayes-risk decoding ranks candidates by executed agreement [19]. CodeLeWM is closest to energy-based scoring: it ranks candidate after-states by a latent transition energy from a JEPA-style predictor, but, unlike test-, execution-, or verifier-based rerankers, it never executes candidate code on the scoring path. Our negative and saturation findings argue that such latent scorers must be measured against no-action, lexical, and shuffled-action controls before any reranking claim is credible.

8 Reproducibility

The source repository is <https://github.com/AbdelStark/CodeLeWM>. The main artifact reports used in this paper are:

- docs/benchmark/V0_2_ACTION_SWAP_HF_RESULTS_2026-05-20.md
- docs/benchmark/EXECUTION_V0_8_RESULTS_2026-06-05.md
- docs/benchmark/EXECUTION_V0_9_RESULTS_2026-06-07.md
- docs/benchmark/PAPER_DEMO_V1_0_ARTIFACTS_2026-06-08.md
- docs/benchmark/V1_0_FINAL_CLAIM_AUDIT_2026-06-08.md
- docs/benchmark/SHUFFLED_ACTION_CONTROL_2026-06-09.md
- docs/benchmark/v1_0/paper_demo/manifest.json

The deterministic replay command for the final demo is:

```
uv run scripts/paper-demo --out docs/benchmark/v1_0/paper_demo --overwrite --json
```


The paper build command is:

```
scripts/build-codelewm-final-paper
```

The final release reproducibility checklist is `docs/release/V1_0_REPRODUCIBILITY_CHECKLIST_2026-06-08.md`.

9 Limitations

- The final positive result is a slice result, not a broad benchmark result.
- The final paper demo replays checked-in v0.9 score rows over a single deterministic scoring pass per seed on the same problems. It is reproducible and manifest-backed, but it is not a fresh checkpoint scoring run, and we quantify only a paired problem-level bootstrap; artifact reproducibility is not robustness of the effect.
- The one open slice is small ($n = 47$, a few problems of headroom) and its lift over the strongest control, shuffled-action, has a 95% CI that includes zero; we report it as suggestive and non-aggregated, not as a robust effect.
- MBPP-Plus is saturated on its 17 problems, so it has no headroom to measure any lift; this is non-informativeness, not measured equivalence.
- The representation probes do not support naming semantic dimensions.
- Broad semantic surprise remains blocked by scorable-pair coverage.
- Candidate code remains untrusted; the model-scoring path does not execute candidate patches.
- The results do not compare against large pretrained code models as generators. CodeLeWM is a scorer/reranker artifact.

10 Outlook: What Would Make the Next Claim Stronger?

The next credible positive claim should be designed around gates that would change the conclusion, not around a more polished demonstration surface. We see five concrete directions.

Action-discriminative data. The training data should contain same-before and near-before candidate sets where the action is necessary to choose the target. This directly attacks the v0.2 no-action failure.

Non-saturated downstream labels. The next claim needs non-saturated downstream labels: at least 100 public-safe labeled tasks per target slice, with no-action and lexical controls below saturation, and lift required over no-action, lexical, shuffled-action, and LLM-order with confidence intervals excluding zero. We have already implemented and released this instrument rather than merely proposing it (schema `codelewm.downstream_anti_saturation_report.v1`): a deterministic public-safe hard-negative pool generator (no-action baits, partial fixes, wrong-symbol or wrong-branch fixes, over-broad fixes, deterministic semantic mutants, and LLM-generated valid candidates), LLM candidate-pack ingestion, a `downstream-rerank -hard-mode` scorer over ten baselines with bootstrap lift CIs, and a `hard-downstream-publish` path with manifest, secret scan,

and claim audit. It applies a three-baseline gate: a slice opens a positive claim only if CodeLeWM beats no-action, lexical, *and* LLM-order on both pass@1 and MRR with all three bootstrap lift CIs excluding zero, on a locked test slice of at least 100 problems. No eligible locked slice has yet been scored, so this gate currently *remains closed*; we release the harness as a falsification instrument, not as evidence of any new positive result.

First-class calibrated scoring heads. If `p_pass` is intended to support downstream ranking, it should be serialized as a distinct score key in every downstream row, calibrated on held out tasks, and evaluated separately from energy-derived CodeLeWM scores.

Representation probes tied to executable semantics. Semantic latent-axis claims should require probes that beat before-state, no-action, lexical, and direct `z_after` controls across seeds. Broad semantic surprise should require enough scorable pairs to make the gate meaningful.

Live utility only after offline gates. LLM patch demos are useful workflow evidence, but live utility claims should come after offline gates pass. Candidate code should remain untrusted, and any execution should stay inside explicit sandbox allowlists with manifests and secret scans.

11 Conclusion

CodeLeWM is a completed reproducible research artifact, not a broad coding assistant. The artifact discipline worked: datasets, checkpoints, reports, claim gates, manifests, and scans can be regenerated and inspected. The original action-use hypothesis did not work: no-action stayed stronger on commit-edit retrieval. The execution substrate gave healthier model diagnostics and a single narrow HumanEval WS-D downstream win that beats no-action and the stronger shuffled-action control by a few problems, but MBPP-Plus saturation, a shuffled-action lift whose confidence interval includes zero, and representation blockers keep the public model-quality claim closed. The durable contribution is methodological: code world-model reranking claims should be published with pre-registered gates and their no-action, lexical, and shuffled-action controls intact—and, where the gate stays closed, with a released falsification instrument rather than a louder claim.

References

- [1] Jordi Armengol-Estap, Quentin Carbonneaux, Tianjun Zhang, Aram H. Markosyan, Volker Seeker, Chris Cummins, Melanie Kambadur, Michael F. P. O’Boyle, Sida Wang, Gabriel Synnaeve, and Hugh James Leather. What i cannot execute, i do not understand: Training and evaluating llms on program execution traces, 2025. URL <https://arxiv.org/abs/2503.05703>.
- [2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. URL <https://arxiv.org/abs/2301.08243>.
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.

- [4] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. URL <https://arxiv.org/abs/2511.08544>.
- [5] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning, 2022. URL <https://arxiv.org/abs/2105.04906>.
- [6] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL <https://arxiv.org/abs/2404.08471>.
- [7] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests, 2022. URL <https://arxiv.org/abs/2207.10397>.
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [9] Yangruibo Ding, Ben Steenhoeck, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. Traced: Execution-aware pre-training for source code, 2023. URL <https://arxiv.org/abs/2306.07487>.
- [10] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages, 2020. URL <https://arxiv.org/abs/2002.08155>.
- [11] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. Graphcodebert: Pre-training code representations with data flow, 2021. URL <https://arxiv.org/abs/2009.08366>.
- [12] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unixcoder: Unified cross-modal pre-training for code representation, 2022. URL <https://arxiv.org/abs/2203.03850>.
- [13] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps, 2021. URL <https://arxiv.org/abs/2105.09938>.
- [14] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning, 2022. URL <https://arxiv.org/abs/2207.01780>.

- [15] Yujia Li, David Choi, Junyoung Chung, et al. Competition-level code generation with alpha-code, 2022. URL <https://arxiv.org/abs/2203.07814>.
- [16] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023. URL <https://arxiv.org/abs/2305.01210>.
- [17] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels, 2026. URL <https://arxiv.org/abs/2603.19312>.
- [18] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks, 2021. URL <https://arxiv.org/abs/2105.12655>.
- [19] Freda Shi, Daniel Fried, Marjan Ghazvininejad, Luke Zettlemoyer, and Sida I. Wang. Natural language to code translation with execution, 2022. URL <https://arxiv.org/abs/2204.11454>.
- [20] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022. URL <https://arxiv.org/abs/2203.11171>.
- [21] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 2021. URL <https://arxiv.org/abs/2109.00859>.
- [22] Wojciech Zaremba and Ilya Sutskever. Learning to execute, 2015. URL <https://arxiv.org/abs/1410.4615>.
- [23] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction, 2021. URL <https://arxiv.org/abs/2103.03230>.
- [24] Tianyi Zhang, Tao Yu, Tatsunori B. Hashimoto, Mike Lewis, Wen tau Yih, Daniel Fried, and Sida I. Wang. Codex reviewer reranking for code generation, 2022. URL <https://arxiv.org/abs/2211.16490>.